Design and Implementation of a Rule-Based Scholarship Recommendation System

Kshitij Garg
Dept. of Computer Science
MIT World Peace University
Pune, India
kshitijvgarg@gmail.com

Ayush Jannawar
Dept. of Computer Science
MIT World Peace University
Pune, India
ayushjannawar@gmail.com

Rishab Kaundinya Thumati
Dept. of Computer Science
MIT World Peace University
Pune, India
rishabkaundinya2004@gmail.com

Abstract*—The distribution of scholarships in higher education institutions often relies on manual verification of student eligibility against multiple criteria, a process that is time-consuming and prone to human error. This paper presents the design and implementation of a Rule-Based Scholarship Recommendation System developed in Java. The application uses a Java Swing graphical user interface paired with an embedded SQLite database to provide a self-contained, offline desktop tool for scholarship eligibility determination. The system implements a rule engine that evaluates student profiles against configurable eligibility criteria including minimum CGPA, maximum family income, minimum attendance percentage, maximum backlogs, category reservation, gender, disability status, and state of domicile. Scholarships and their associated rules are stored as data in the database, allowing administrators to create, modify, and delete eligibility criteria without altering application code. The system provides CRUD operations for both student profiles and scholarship definitions, a single-student or batch eligibility check, and a tabular report view displaying results across all student-scholarship combinations. The application is packaged as a lightweight desktop tool requiring only a Java runtime and an SQLite JDBC driver, making it suitable for deployment in resource-constrained administrative offices.

***Keywords*—Scholarship Recommendation, Rule Engine, Java Swing, Desktop Application, SQLite Database, Data Access Object, Eligibility Determination**

I. Introduction

Educational scholarships serve as a critical financial support mechanism for students pursuing higher education. In many institutions, particularly in India, scholarship eligibility determination is performed manually by administrative staff who cross-reference student profiles against printed eligibility handbooks. As the number of students and available scholarships grows, this manual process becomes increasingly burdensome and error-prone.

The primary challenges with manual eligibility evaluation include: the time required to check each student against every available scholarship, the risk of arithmetic or transcription errors when comparing numeric values like CGPA against thresholds, and the difficulty of maintaining consistent decision records for audit or appeal purposes.

This project addresses these challenges by implementing a Rule-Based Scholarship Recommendation System. The application digitizes eligibility criteria into configurable rules stored in a relational database and evaluates student data against these rules programmatically. The system is designed as a self-contained, offline desktop application using Java Swing for the user interface and SQLite for data persistence.

The contributions of this paper are as follows:

- A layered architecture separating the user interface, business logic, and data persistence, with the Data Access Object (DAO) pattern mediating database interactions.
- A rule engine that models eligibility criteria as data entities rather than hardcoded conditional branches, supporting nine distinct rule types and seven comparison operators.
- A Java Swing desktop application with four tabbed panels for student management, scholarship management, eligibility checking, and report generation.
- Use of parameterized SQL queries (PreparedStatement) throughout the DAO layer to prevent SQL injection vulnerabilities.
- A sample data initializer that seeds the database with representative student profiles and scholarship definitions for demonstration purposes.

The remainder of this paper is organized as follows. Section II surveys related work. Section III describes the system requirements. Section IV elaborates the system architecture and database schema. Section V presents the rule engine design. Section VI covers implementation details. Section VII discusses limitations, Section VIII concludes, and Section IX outlines future work.

II. Related Work

The problem of automated eligibility determination touches several areas of computer science, including rule engines, recommender systems, and administrative desktop software.

A. Rule Engines in Software Systems

Production-grade rule engines such as Drools and JESS have been widely used in enterprise domains including insurance underwriting and loan origination. These systems use algorithms like RETE to efficiently evaluate large numbers of rules against fact bases. However, for the bounded cardinalities typical of a university context—a few thousand students and a few dozen scholarships—the engineering overhead of integrating a general-purpose rule engine outweighs its benefits. Our system implements a simpler, direct-evaluation approach where each rule is evaluated independently against a student's data, which is straightforward to understand and maintain.

B. Desktop vs. Web Administrative Tools

While web-based administrative tools offer advantages in multi-user environments, a significant fraction of Indian higher education administration occurs in offices with intermittent internet connectivity and limited IT support. A self-contained Java desktop application that runs from compiled class files with an embedded SQLite database

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

provides a qualitatively different reliability profile: it functions without network connectivity and imposes no server administration cost.

C. Gap Addressed by This Work

This project fills a practical gap by providing a lightweight, offline-capable tool that treats scholarship eligibility as a deterministic constraint-satisfaction problem. The system is scoped for a single-administrator, single-institution deployment and is implementable by an undergraduate team within a semester-long project.

III. System Requirements and Objectives

Prior to designing the system architecture, a requirements analysis was conducted to identify the needs of scholarship administrators.

A. Functional Requirements

The system must support Create, Read, Update, and Delete (CRUD) operations for two primary domain entities: Student Profiles and Scholarships. Each Scholarship is associated with one or more Eligibility Rules, which are managed as part of the scholarship definition. The application must provide a mechanism to evaluate a selected student against a selected scholarship and display the eligibility verdict with a breakdown of passed and failed criteria. Additionally, the system must support batch evaluation, checking a single student against all scholarships or generating a comprehensive report of all students against all scholarships.

B. Non-Functional Requirements

The primary non-functional requirement is operational portability: the system must operate without requiring an external database server and without internet connectivity. The application should provide immediate visual feedback for user actions through standard Swing dialog messages. Input validation should reject malformed data at the UI layer before it reaches the database. The system should use parameterized SQL queries to prevent injection attacks.

IV. System Architecture

The system is organized into a layered architecture with four packages under the `com.scholarship` namespace: `model` for data-carrier classes, `dao` for database access, `service` for business logic, and `ui` for the presentation layer. The application entry point is the `App` class, which sets the system look-and-feel and launches the `MainFrame` on the Swing Event Dispatch Thread (EDT).

TABLE I. Database Schema Overview

Table Name	Schema Properties		
	Primary Key	***Core Attributes***	***Foreign Keys***
students	id (AUTOINCREMENT)	name, roll_number, email, cgpa, family_income, category, attendance_percent, backlog_count, date_of_birth, gender, is_disabled, state, current_semester	None
scholarships	id (AUTOINCREMENT)	name, description, amount, provider, is_active	
eligibility_rules	id (AUTOINCREMENT)	rule_type, operator, value, description, is_mandatory	scholarship_id → scholarships(id)
eligibility_results	id (AUTOINCREMENT)	is_eligible, passed_rules, failed_rules, remarks, checked_at	student_id → students(id), scholarship_id → scholarships(id)

[a] All four tables use INTEGER PRIMARY KEY AUTOINCREMENT. Foreign key references are declared but SQLite does not enforce them by default unless PRAGMA foreign_keys is enabled.

Fig. 1. High-level block diagram of the layered architecture and data flow across the packages.

A. Presentation Layer (Java Swing)

The user interface is built using Java Swing and is organized as a `JFrame` (`MainFrame`) containing a `JTabbedPane` with four tabs. The `StudentPanel` provides a form for entering student data and a `JTable` displaying all students. The `ScholarshipPanel` provides a form for scholarship details along with inline fields for configuring eligibility rules (minimum CGPA, maximum income, minimum attendance, maximum backlogs, and category). The `EligibilityPanel` uses `JComboBox` dropdowns to select a student and scholarship, with buttons to check eligibility for a single combination or for all scholarships. The `ReportPanel` generates a tabular report of all student-scholarship eligibility results in a `JTable`. All tables use `DefaultTableModel` with cell editing disabled. A `JMenuBar` provides File (Exit) and Help (About) menus.

B. Business Logic Layer

The service layer resides in the `com.scholarship.service` package and contains two classes. The `RuleEngine` class implements the core evaluation logic: given a `Student` and an `EligibilityRule`, it extracts the relevant student attribute, applies the rule's operator and threshold, and returns a boolean result. The `EligibilityService` class orchestrates rule evaluation by iterating over all rules associated with a scholarship, collecting pass/fail descriptions, and producing an `EligibilityResult` object. It also provides methods to check all scholarships for a given student and to compute summary statistics (total, eligible, not eligible counts).

C. Persistence Layer (SQLite)

The system uses an embedded SQLite database (`scholarship.db`) accessed via the `org.sqlite.JDBC` driver (version 3.42.0.0, included in the `lib/` directory). The `DatabaseConnection` class provides a static `getConnection()` method that returns a new connection per call via `DriverManager.getConnection()`. The `initializeDatabase()` method creates the four tables using `CREATE TABLE IF NOT EXISTS` statements. Two DAO classes, `StudentDAO` and `ScholarshipDAO`, encapsulate all SQL operations using `PreparedStatement` for parameterized query execution. The `ScholarshipDAO` also handles CRUD operations for eligibility rules, inserting rules when a scholarship is created and replacing all rules on update (delete-then-reinsert).

D. Data Model

The data model consists of four classes in the `com.scholarship.model` package. `Student` is a POJO with fields for id, name, roll number, email, CGPA, family income, category, attendance percentage, backlog count, date of birth, gender, disability status, state, and current semester. `Scholarship` holds an id, name, description, amount, provider, active flag, and a `List<EligibilityRule>`. `EligibilityRule` contains an id, scholarship ID, a `RuleType` enum, an `Operator` enum, a string value, description, and a mandatory flag. `EligibilityResult` aggregates a student, a scholarship, lists of passed and failed rule descriptions, a timestamp, and a boolean eligibility verdict (determined by checking if the failed-rules list is empty).

V. Algorithmic Design: The Rule Engine

The core of the system is the `RuleEngine` class, which evaluates whether a student satisfies a given eligibility rule. Rules are modeled as data entities stored in the database rather than as hardcoded conditional branches, which allows administrators to modify eligibility criteria without changing the application code.

An `EligibilityRule` links a `RuleType` (one of nine enumerated types: `MIN_CGPA`, `MAX_INCOME`, `MIN_ATTENDANCE`, `CATEGORY`, `MAX_BACKLOGS`, `GENDER`, `DISABILITY`, `STATE`, `MIN_SEMESTER`) with an `Operator` (one of seven: `GREATER_THAN`, `LESS_THAN`, `GREATER_EQUAL`, `LESS_EQUAL`, `EQUALS`, `NOT_EQUALS`, `IN_LIST`) and a threshold value stored as a string.

For a given scholarship with N associated rules, the eligibility verdict is the logical conjunction of all individual rule results. The `EligibilityService` iterates over every rule, and the `EligibilityResult.determineEligibility()` method sets the verdict to true only if the failed-rules list is empty. If any single rule fails, the student is deemed ineligible for that scholarship.

A. Numeric Rule Evaluation

For numeric rule types (`MIN_CGPA`, `MAX_INCOME`, `MIN_ATTENDANCE`, `MAX_BACKLOGS`, `MIN_SEMESTER`), the engine extracts the corresponding numeric value from the student object via a `switch` expression on the `RuleType`. The rule's threshold string is parsed to a `double`, and the comparison is performed according to the operator. A `NumberFormatException` during parsing is caught and defaults the threshold to 0.0.

B. String and Categorical Rule Evaluation

For string-based rule types (`CATEGORY`, `GENDER`, `STATE`), the engine extracts the student's string attribute. The `DISABILITY` rule type converts the boolean `isDisabled` flag to the string "YES" or "NO". The `EQUALS` and `NOT_EQUALS` operators perform case-insensitive string comparison. The `IN_LIST` operator splits the rule's comma-separated threshold value, trims and uppercases each token, and checks whether the student's value (also uppercased) matches any token. This case-insensitive matching prevents inconsistencies from administrator data entry.

C. Rule Description Generation

The `RuleEngine` also provides a `getRuleDescription()` method that generates a human-readable string for each rule (for example, "CGPA ≥ 8.5" or "Category in [SC,ST]"). These descriptions are collected in the `EligibilityResult` as passed or failed criteria and displayed to the user in the eligibility check output.

VI. Implementation Details

This section covers specific engineering choices made during implementation.

Fig. 2. Screenshot of the Scholarship Eligibility Automation System showing the tabbed interface.

A. Application Startup and Sample Data

The `App.main()` method sets the system look-and-feel via `UIManager.setLookAndFeel()` and launches the `MainFrame` on the EDT using `SwingUtilities.invokeLater()`. During `MainFrame` initialization, `DatabaseConnection.initializeDatabase()` creates the database tables if they do not exist, and `SampleDataInitializer.initializeSampleData()` checks whether the `students` table is empty. If it is, the initializer inserts five sample student profiles and five sample scholarship definitions with realistic Indian demographic data and rule configurations modeled after actual government scholarship schemes (INSPIRE, Post-Matric SC/ST, Central Sector Scheme, Pragati Scholarship for Girls, OBC Pre-Matric Scholarship).

B. User Interface Event Handling

User interactions are handled via `ActionListener` implementations attached to buttons. The `StudentPanel` and `ScholarshipPanel` each provide Add, Update, Delete, and Clear Form buttons. Input validation in these panels checks for required fields (name, roll number for students; name, amount, provider for scholarships) and catches `NumberFormatException` for numeric fields, displaying error dialogs via `JOptionPane`. Table row selection triggers loading of the selected record into the form fields via `ListSelectionListener`. The `EligibilityPanel` uses a `HierarchyListener` to refresh its combo box contents whenever the panel becomes visible, ensuring that newly added students and scholarships appear in the dropdowns.

C. Parameterized Queries

Every SQL statement in the DAO layer is constructed via `PreparedStatement` with typed parameter binding. This prevents SQL injection by ensuring that user-supplied data is never concatenated into SQL text. Each DAO method acquires a fresh `Connection` from `DatabaseConnection.getConnection()`, performs the operation within a try-with-resources block, and releases the connection on completion.

D. Scholarship Rule Management

The `ScholarshipPanel` exposes rule configuration through dedicated form fields for minimum CGPA, maximum income, minimum attendance, maximum backlogs, and a category dropdown. When a scholarship is added or updated, non-empty rule fields are converted into `EligibilityRule` objects and attached to the `Scholarship`. On update, the `ScholarshipDAO` deletes all existing rules for the scholarship and reinserts the new set, implementing a full-replacement strategy.

E. Report Generation

The `ReportPanel` provides a "Generate Report" button that performs a bulk evaluation: it retrieves all students and all scholarships from the database, evaluates every student-scholarship pair, and populates a `JTable` with columns for student name, roll number, scholarship name, eligibility verdict (Yes/No), number of passed rules, and number of failed rules. This provides an at-a-glance overview of eligibility across the entire institution.

VII. Limitations

The present implementation has several limitations that should be understood by adopters.

- **No background threading.** All database operations and rule evaluations run on the Swing EDT. For large datasets, this could cause the UI to become unresponsive during report generation.
- **No audit trail persistence.** While the database schema includes an `eligibility_results` table, the current application does not write evaluation results to it. Results are computed on-the-fly and displayed in the UI but not persisted for later audit.
- **No CSV import or export.** Student data must be entered manually through the UI form; there is no facility to import from or export to CSV files.
- **Single-writer concurrency.** SQLite enforces single-writer semantics, limiting the system to a single-administrator deployment.
- **Limited input validation.** Validation checks for required fields and numeric format, but does not enforce range constraints (for example, CGPA between 0 and 10) or cross-field consistency.
- **No build system.** The project is compiled directly with `javac` and does not use Maven, Gradle, or any build automation tool.
- **No automated tests.** The project does not include a test suite.
- **Rule types limited to UI.** While the `EligibilityRule.RuleType` enum includes nine types, the `ScholarshipPanel` form only exposes five of them (MIN_CGPA, MAX_INCOME, MIN_ATTENDANCE, MAX_BACKLOGS, CATEGORY). Rules for GENDER, DISABILITY, STATE, and MIN_SEMESTER can only be created programmatically (as done in the sample data initializer).
- **Foreign key enforcement.** SQLite does not enforce foreign keys by default, and the application does not execute `PRAGMA foreign_keys = ON`, so referential integrity is not guaranteed at the database level.

VIII. Conclusion

This paper has presented the design and implementation of a Rule-Based Scholarship Recommendation System built with Java Swing and SQLite. The system digitizes scholarship eligibility criteria into configurable database-stored rules and evaluates student profiles against them programmatically, replacing the manual cross-referencing workflow. The layered architecture separates the presentation, business logic, and persistence concerns, with the DAO pattern providing parameterized database access. The rule engine supports nine rule types and seven comparison operators, covering academic, demographic, and financial eligibility criteria. The application provides CRUD interfaces for students and scholarships, single and batch eligibility checking, and a comprehensive report view. As a self-contained desktop application requiring only a Java runtime, it is suitable for deployment in environments without reliable internet connectivity or dedicated IT support.

IX. Future Scope and Expansions

The system establishes a foundation for several future enhancements:

- **Background threading.** Delegating batch evaluation to `SwingWorker` threads would keep the UI responsive during report generation over large datasets.
- **Audit log persistence.** Writing evaluation results to the `eligibility_results` table would provide a persistent audit trail for post-hoc review and appeal processing.
- **CSV import and export.** Adding CSV import for student rosters and CSV export for evaluation results would integrate the system with existing administrative data workflows.
- **Full rule-type UI support.** Extending the `ScholarshipPanel` form to expose all nine rule types would eliminate the need for programmatic rule creation.
- **Input validation enhancement.** Adding range validation (CGPA 0–10, attendance 0–100) and enabling SQLite foreign key enforcement via `PRAGMA foreign_keys = ON` would improve data integrity.
- **Web-based interface.** Replacing the Swing UI with a web frontend would enable multi-user access and remote administration.
- **Build automation and testing.** Adopting Maven or Gradle for build management and adding unit tests for the rule engine would improve maintainability and correctness assurance.

Acknowledgment

The authors would like to express their appreciation to Dr. Swati Jadhav, our course professor at MIT World Peace University, for her guidance and constructive feedback throughout the development of this project.

References

The following citations represent works covering software design patterns, Java development, relational databases, and desktop application development.

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software." Addison-Wesley, 1994.

[2] D. Flanagan, "Java in a Nutshell: A Desktop Quick Reference," 7th ed. O'Reilly Media, 2018.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, "Database System Concepts," 7th ed. McGraw-Hill Education, 2019.

[4] J. Bloch, "Effective Java," 3rd ed. Addison-Wesley, 2017.

[5] R. Elmasri and S. B. Navathe, "Fundamentals of Database Systems," 7th ed. Pearson, 2015.

[6] D. Hipp, "SQLite: Small. Fast. Reliable. Choose any three," SQLite Consortium Technical Report, 2020.

[7] M. Fowler, "Patterns of Enterprise Application Architecture." Addison-Wesley, 2002.

[8] R. C. Martin, "Clean Architecture: A Craftsman's Guide to Software Structure and Design." Prentice Hall, 2017.

[9] Ministry of Education, Government of India, "National Scholarship Portal Guidelines," 2023.